

AgentID

Master Build Tracker

Complete record of what has been built, where files live, and what comes next.

Last updated: May 2026 · Server v0.8.0 / SDK v0.6.0 · Generated 2026-05-11 20:37

System Architecture

AgentID is an identity, messaging, and autonomy layer for AI agents. Three repositories form the full system:

Repository	Purpose	Location
agentid-pro	Production server (Railway) REST API, auth, messaging, MCP server	/Users/bereket/Documents/agentid-pro/
agentid (SDK)	Python SDK published to PyPI Runtime, Brain, Integrations	/Users/bereket/Documents/agentid/sdk/python/
agentid (docs)	Web dashboard, HTML docs	/Users/bereket/Documents/agentid/docs/

What Has Been Built

Feature	Description	File(s)	Status
Diagnostic logging	Added 9 silent failure logs to <code>_auto_respond_bg</code>	agentid-pro/server_pro.py	DONE
Anthropic auto-reply	Added Anthropic as second provider alongside OpenAI. <code>_provider_handled</code> flag prevents double-replies.	agentid-pro/server_pro.py lines ~870-930	DONE
MCP Sampling (Claude Desktop)	When Claude Desktop is open, it generates auto-replies using its full session context. Falls back to API key when closed. Priority: MCP sampling → Anthropic → OpenAI.	agentid-pro/mcp_server.py agentid-pro/server_pro.py	DONE

Feature	Description	File(s)	Status
AgentRuntime + long-polling	Persistent polling loop with <code>MessageContext</code> , concurrent handlers, health endpoint on <code>:8080</code> , cursor persistence.	agentid/sdk/python/agentid/runtime/runtime.py, client.py, config.py, handler.py	DONE
Webhook transport	Alternative to long-polling. Brain registers a public URL; server POSTs messages instantly. HMAC-SHA256 signature verification.	agentid/sdk/python/agentid/runtime/webhook.py	DONE
CLI (agentid-runtime)	<code>python3 -m agentid.runtime --config agentid.toml --handler app:handle</code> Flags: <code>--mode poll webhook</code> , <code>--webhook-url</code> , <code>--webhook-port</code>	agentid/sdk/python/agentid/runtime/_main_.py	DONE

Feature	Description	File(s)	Status
Perception layer	GitPerception (repo diffs), FilePerception (SHA-256 checksums), APIPerception (HTTP endpoint change detection).	agentid/sdk/python/agentid/brain/perception/ base.py, git.py, files.py, api.py	DONE
Judgment engine	LLM-powered decision making. Returns structured JSON: should_act, reasoning, summary, actions[].	agentid/sdk/python/agentid/brain/judgment/engine.py	DONE
Action executor	Executes send_message, find_and_contact, alert_owner, store_note via AgentID network.	agentid/sdk/python/agentid/brain/actions/executor.py	DONE
BrainMemory	Persistent JSON state per agent. Stores perception state tokens, action history (cap 200), LLM-accumulated notes.	agentid/sdk/python/agentid/brain/memory/store.py	DONE
Triggers	IntervalTrigger (every N seconds), DailyTrigger (UTC time), OnChangeTrigger (fires only when perception source changes).	agentid/sdk/python/agentid/brain/triggers/ schedule.py, change.py	DONE
AgentBrain orchestrator	Connects all layers. Fluent API: brain.add_perception().add_trigger(). think_once() for single cycle. run() for autonomous loop.	agentid/sdk/python/agentid/brain/brain.py	DONE

Feature	Description	File(s)	Status
agent_task_requests table + schema init	New table agent_task_requests (NOT agent_tasks — that name is taken). Columns: task_id (uuid4), from_did, to_did, capability, contract_version, inputs (JSONB), output (JSONB), status (CHECK constraint), deadline_at, artifact_file_ids, sender_owner, receiver_owner, idempotency_key. Partial indexes for polling, deadline sweeping, idempotency dedup.	agentid-pro/agent_task_requests.py	DONE
5 REST endpoints (no config required)	POST /agents/{did}/tasks — capability contracts optional: if one exists its schema is validated (400 on mismatch), otherwise task proceeds with version '1.0'. No require_pro() gate — free tier included. GET /agents/{did}/tasks/{id} (poll with ?wait=N long-poll), GET /agents/{did}/tasks (list with role/status/capability filters), PATCH /agents/{did}/tasks/{id} (state transitions, atomic WHERE), DELETE /agents/{did}/tasks/{id} (cancel pending only).	agentid-pro/agent_task_requests.py	DONE

Atomic state machine	All transitions enforced via UPDATE...WHERE status=current_status RETURNING. 0 rows = race condition → 409. No SELECT-then-UPDATE pattern. States: pending→accepted→running→completed failed rejected expired cancelled.	agentid-pro/agent_task_requests.py _VALID_TRANSITIONS	DONE
jsonschema input/output validation	Inputs validated against capability_contracts.input_schema at submit time (synchronous 400, not deferred). Output validated against output_schema at PATCH completion. Empty schema = accept anything.	agentid-pro/agent_task_requests.py _validate_schema()	DONE
Capability auto-routing	Omit to_did → platform finds best agent by capability + trust score. SQL ranking: verified > pending_verification > challenge_passed, then sla_compliance_score, then attestation_score. Python scoring: 40% status + 35% SLA + 25% attestation.	agentid-pro/agent_task_requests.py _find_best_capability_match()	DONE
Expiry sweeper + 6 webhook events	sweep_expired_tasks() hooked into _global_proactive_scheduler (every 180s). Webhook events: task.created→receiver, task.accepted/completed/failed/expired/rejected→sender. VALID_EVENTS extended in webhooks.py.	agentid-pro/agent_task_requests.py sweep_expired_tasks() agentid-pro/webhooks.py	DONE
Brain execute_task action	_brain_execute_task(): full tool loop (Anthropic + OpenAI, 6 rounds). Brain injects pending task requests as context each cycle. execute_task action added to brain decision schema. _auto_respond_bg skips task_notification messages. require_agent_ownership() added to auth.py.	agentid-pro/server_pro.py _brain_execute_task() agentid-pro/auth.py require_agent_ownership()	DONE

Feature	Description	File(s)	Status
6-dimension decomposition	Reorganised all existing flat signals into 6 dimension vectors (each 0-100): D1 Identity Integrity, D2 Operational Reliability, D3 Network Reputation, D4 Behavioral History, D5 Governance, D6 Capability Trust. No new data required — all computed from existing signals in trust_score.py.	agentid-pro/trust_score.py lines 322-345	DONE
Compromised feed hard-pin (D1)	trust_compromised table check: if quorum_reached=TRUE the agent's Identity Integrity dimension is hard-pinned to 0 regardless of other signals.	agentid-pro/trust_score.py is_compromised check	DONE

Badge graduated (D5)	Badge score changed from binary cliff (0 or 25 pts on composite) to graduated in D5: badge = 50 governance pts, contracts = up to 50 pts. Full badge credit requires compliance mode (Phase 2). Existing composite badge_score unchanged — backward compatible.	agentid-pro/trust_score.py d5_score formula	DONE
Per-capability trust map (D6)	New DB query groups capability_attestations by capability tag. Each capability gets an independent 0-100 score. Result: capability_trust: {research: 74, code: 12, summarize: 88}	agentid-pro/trust_score.py capability_trust block	DONE
LLM Trust Brief (_generate_trust_brief)	Server-side narrative string. Formats dimensions, capability trust, auto-generated red flags (compromised, no governance, unreliable, weak capabilities), and guidance (safe for / caution for). Returned in trust_brief field on all authenticated endpoints.	agentid-pro/trust_score.py _generate_trust_brief()	DONE
API response dimensions field	Public endpoint: score+level+dimensions. Pro endpoint: full breakdown+dimensions+trust_brief. Pro list: dimensions per agent. All backward-compatible.	agentid-pro/trust_score.py 3 route handlers	DONE

Feature	Description	File(s)	Status
Complaint density + resolution rate (D3)	Query agent_complaints 90d window. complaint_density_penalty=-3/complaint (max-20), resolution_bonus up to +10. complaint_net*0.5 feeds D3. complaint_density breakdown field with 90d stats.	agentid-pro/trust_score.py complaint_density_penalty block	DONE
Graduated badge D5 (tier 1/2/3)	Badge score now tiered: 0=no badge, 15=badge only (tier1), 20=badge+contracts (tier2), 25=badge+strong governance+clean record (tier3). badge_gov proportional mapping to D5 (0→0, 15→30, 20→40, 25→50). Replaces old binary cliff.	agentid-pro/trust_score.py graduated badge block	DONE
Signing compliance rate (D1)	Query agent_messages outbound non-auto 30d. 90%+ signed=10pts, 70-90%=7, 50-70%=4, <50%=1, neutral=5 (no msgs). D1 raw max raised 25→40.	agentid-pro/trust_score.py signing_compliance block	DONE
Key rotation history table + D1 bonus	key_rotation_history table created in key_rotation.py init. confirm_rotation logs completed rotations. rotation_bonus 0-5 pts in D1.	agentid-pro/key_rotation.py init + confirm_rotation	DONE

Decay-weighted D4 interactions (90d)	SQL fetches (delta, days_ago) for trust_score_events 90d window. Weights: 0-7d=1.0×, 7-30d=0.7×, 30-60d=0.4×, 60-90d=0.2×. Replaces flat 30d window.	agentid-pro/trust_score.py interaction_score block	DONE
Task completion rate (D2)	Query agent_task_requests cross-owner completed vs failed 90d. Confidence-scaled: <3 tasks=partial, 10+=full. +10 pts max to D2. D2 raw max raised 60→70.	agentid-pro/trust_score.py task_completion_score block	DONE
Scope violation tracking (D4)	_record_scope_violation() helper in server_pro.py writes event_type='scope_violation' delta=-5. trust_score.py queries these events, applies -5/violation (90d, max-15) as scope_violation_penalty in D4.	agentid-pro/server_pro.py _record_scope_violation agentid-pro/trust_score.py scope_violation_penalty	DONE
Scope limits registration (3.6)	scope_limits JSONB + human_oversight TEXT columns on agents table. RegisterRequest and UPDATE endpoint accept and persist them. DID document now includes #scope-declaration service endpoint. D5: scope_gov 0-15 pts based on scope+oversight declared.	agentid-pro/server_pro.py scope_limits columns + get_scope_info agentid-pro/did_document.py #scope-declaration	DONE
DID-document consistency check (3.2)	Derives expected DID from stored public_key via public_key_to_did. Mismatch hard-pins D1=0 (same severity as quorum compromise). did_consistency breakdown field, CRITICAL red flag in trust brief.	agentid-pro/trust_score.py did_consistent check	DONE
Trust brief injection across MCP tools	check_trust_for_action and get_trust_score now inject trust_context brief. Public /agents/{did}/trust-score now includes trust_brief field. All 4 trust-decision MCP tools now surface rich context inline.	agentid-pro/mcp_server.py check_trust_for_action + get_trust_score	DONE
dimension_snapshots dict-unwrap fix	Dimension values from _compute_trust_score are dicts {score:X, trend_30d:...}. _extract_dim_score() helper extracts float before storing in FLOAT DB column. Prevented silent None insertions in daily snapshot batch.	agentid-pro/dimension_snapshots.py _extract_dim_score	DONE

Feature	Description	File(s)	Status
---------	-------------	---------	--------

pagerank.py new module	agent_pagerank table stores normalised score per DID. run_pagerank() builds graph from capability_attestations (cross-owner only). Power-iteration with damping=0.85, max 30 iters, convergence=1e-6. Edge weights: verdict (confirmed=1.0×, partial=0.5×) × reviewer_trust_at_time/50. Handles dangling nodes. Normalises final ranks to 0.0-1.0. Upserts results per DID on completion.	agentid-pro/pagerank.py run_pagerank	DONE
D3 Network Reputation: PageRank signal	pagerank_score = get_pagerank_score(did) * 5.0 (0-5 pts). D3 max raw raised 20→25. pagerank breakdown field. Trust brief shows 'Endorsement graph: PageRank X → Y/5 pts' guidance.	agentid-pro/trust_score.py D3 + pagerank block	DONE
Nightly batch + endpoints	server_pro.py: init_pagerank_schema() at startup; run_pagerank() fires nightly after take_all_snapshots() (~25h cycle). GET /agents/{did}/pagerank: score + percentile + batch metadata. POST /internal/pagerank/run: admin/internal trigger.	agentid-pro/server_pro.py nightly batch agentid-pro/pagerank.py endpoints	DONE

Feature	Description	File(s)	Status
_check_polp() helper in capability_contract s.py	Looks up callee's scope_limits from agents table. Empty scope_limits = permissive (no restriction). Exact or prefix match: 'research' scope permits 'research-summary' calls. On out-of-scope: records scope_violation trust event (-5 pts D4) via _record_scope_violation. Fails open on any DB error — never accidentally blocks on transient failure.	agentid-pro/capability_contracts.py _check_polp function	DONE
log_call_endpoint POLP gate (Layer 2)	POLP check runs before logging every capability call. POLP_ENFORCE=1: HTTP 403 block with clear message. POLP_ENFORCE=0 (default, audit-only): allow call + scope_warning in response. scope_warning includes scope_limits list and violation notice.	agentid-pro/capability_contracts.py log_call_endpoint	DONE
GET /agents/{did}/scope-check endpoint	Direct POLP query endpoint. Returns in_scope bool, scope_limits list, polp_enforce bool, violation_recorded bool. Records violation event on out-of-scope queries (caller_did param for audit trail).	agentid-pro/capability_contracts.py check_agent_scope endpoint	DONE
check_agent_scope MCP tool	Read-only MCP tool wrapping /agents/{did}/scope-check. Added to MCP instructions: 'Call check_agent_scope before invoking any agent.' Enables Claude to verify POLP before delegating capability calls.	agentid-pro/mcp_server.py check_agent_scope tool	DONE

Feature	Description	File(s)	Status
90-day uptime tracking (D2)	agent_uptime_days table: one row per (did, day DATE) per UTC day with heartbeat. liveness.py ping handler inserts idempotently. uptime_score = active_days_90d / 90 * 10 (0-10 pts). D2 raw max raised 70→80. uptime_label in breakdown.	agentid-pro/liveness.py agent_uptime_days + ping handler agentid-pro/trust_score.py uptime_score block	DONE
SLA breach penalty from expired tasks (D2)	Expired cross-owner agent_task_requests = hard SLA breach. sla_breach_penalty = -2/breach (90d, max -10). Applied after D2 normalization (can push below 0 before clamp). sla_breach_label shown in breakdown.	agentid-pro/trust_score.py sla_breach_penalty block	DONE
Per-capability 5-signal trust model (D6)	Full D6 model replaces simple attestation-only approach. Signal 1: peer attestations weighted by reviewer_trust_at_time (0-10 pts). Signal 2+3: call log success rate + sla_compliance_score per capability (0-20 pts). Signal 4: task completion rate per capability from agent_task_requests (0-10 pts). Signal 5: per-capability complaints via agent_complaints.category LIKE 'capability:%' (0 to -10 pts). Composite: raw_total/50*100→0-100 per capability. API returns {score, signals:{attestation,call_success, task_completion,complaints}, detail:{call_count, avg_latency_ms,task_rate,open_complaints}}.	agentid-pro/trust_score.py D6 5-signal block	DONE
Per-capability complaint category (accountability.py)	category TEXT column added to agent_complaints (idempotent ALTER TABLE). Category index for fast per-capability lookups. ComplaintIn model accepts optional category (max 64 chars). Callers use 'capability:research' format to tag capability-specific complaints.	agentid-pro/accountability.py category column + index	DONE
Trust brief D6 fix (_generate_trust_brief)	capability_trust values changed from float to {score,signals,detail} dicts. Fixed CAPABILITY TRUST section to extract .score for display and comparisons. Section now shows per-signal breakdown [att=X, call=Y, task=Z, cmp=W]. red-flag loop and safe_caps/weak_caps comparisons all updated with _cap_score() helper.	agentid-pro/trust_score.py _generate_trust_brief	DONE

Feature	Description	File(s)	Status
---------	-------------	---------	--------

EU AI Act Compliance Mode (eu_compliance.py)	Risk tier classification (minimal/limited/high/unacceptable_risk), human review queue with bypass tokens, POLP enforcement, tamper-evident SHA-256 hash chains, monthly PDF compliance report. Risk tier published in agent DID document (#eu-ai-act service entry). 6 endpoints: POST/GET /pro/compliance/eu-ai-act, POST /pro/agents/{did}/risk-tier, review queue CRUD.	agentid-pro/eu_compliance.py agentid-pro/did_document.py	DONE
Task API fixes (no-reply bug)	_notify_task_via_message() now triggers _agent_brain_cycle(to_did) in background thread. _get_unprocessed_messages returns msg_type. Brain renders task_notification with ■ STRUCTURED TASK REQUEST header.	agentid-pro/agent_task_requests.py agentid-pro/server_pro.py	DONE
Tasks.html API key auth fix	/auth/me endpoint now accepts both session cookie AND x-api-key header. Previously API key login on tasks page always failed (cookie-only path).	agentid-pro/accounts.py me() endpoint	DONE
Security audit — 13 findings fixed	CRITICAL: broken admin tier gate → require_enterprise(), SQL f-string injection in trust_routing.py, SSRF redirect bypass in identity_binding.py. HIGH: CORS missing PUT, threading locks for _bypass_tokens/_search_us age/_SESSION_CACHE, circular import made lazy, session only slides when >50% TTL consumed. MEDIUM: _account_age_days → users.created_at, async long-poll (asyncio.sleep), SQL LIMIT/OFFSET f-strings → parameterized.	accountability.py, trust_routing.py, identity_binding.py, server_pro.py, eu_compliance.py, accounts.py, agent_task_requests.py	DONE
Premium dashboard redesign	4px custom scrollbar, stat card sparklines + accent gradient top border, card hover lift (box-shadow + translateY(-1px)), sidebar pill-nav + avatar footer, fadeSlideUp entrance animations, dark mode glass morphism, compact mode CSS. Settings → Preferences tab: 9 toggle switches + 2 selects, localStorage persistence, real-time apply (_applyPrefs).	agentid/docs/dashboard.html agentid/docs/dashboard.js	DONE
Network map physics rewrite	Force-directed simulation: Coulomb repulsion (k=12000), Hooke springs (k=0.028), center gravity, velocity damping (0.78), alpha cooling. Golden angle spiral initial layout. Gradient edges, trust score mini-badges at zoom>0.6, dashed ring for compromised. Detail panel: SVG trust ring, curated risk signals, capability chips, activity sparkline, top 5 connections with trust scores, background prefetch for 20 agents.	agentid/docs/network.js	DONE

Feature	Description	File(s)	Status
Crypto-agility envelope on all sign() functions	sign() in Python SDK, TypeScript SDK, Go SDK, and internal agentid-pro SDKs now returns {algSuite: 'ed25519-sha512-2024', version: 1, params: {}, signature: ""} instead of a bare base64 string. verify() accepts both envelope dict and legacy bare string (90-day backward compat). signLegacy() added to TS SDK for callers that still need bare strings.	agentid/sdk/python/agentid/crypto.py sign() agentid/sdk/typescript/src/crypto.ts sign() + SignatureEnvelope agentid-pro/sdk/python/agentid/crypto.py sign() agentid-pro/sdk/node/src/crypto.ts sign() + SignatureEnvelope	DONE
TypeScript millisecond timestamp bug fix	Date.now() returns milliseconds; Python verify_from_did() uses time.time() (seconds). All TypeScript-signed payloads previously bypassed the staleness check silently. Fixed: Math.floor(Date.now() / 1000) in agent.ts for all timestamp fields.	agentid/sdk/typescript/src/agent.ts timestamp fields agentid-pro/sdk/node/src/agent.ts timestamp fields	DONE
signer field standardised across all SDKs	Python SDK canonical: payload.signer = agent.did. TypeScript SDK was using signer_did — renamed to signer. Go SDK was omitting the field entirely — signer: a.DID added to Sign() payload map. Internal agentid-pro Node SDK aligned. All cross-SDK verify() calls now pass.	agentid/sdk/typescript/src/agent.ts signer_did → signer agentid/sdk/go/agentid.go Agent.Sign() agentid-pro/sdk/node/src/agent.ts signer field	DONE
Path traversal fix in registry._key_path()	Old: did.replace(':', '_') + '.key' allowed did:agentid:../../../../tmp/evil to escape key dir. New: re.sub(r['^a-zA-Z0-9_\-'], '_', did) sanitises all non-safe chars. Then path.resolve().relative_to(key_dir.resolve()) asserts containment (raises ValueError on escape). Applied to both LocalRegistry and FileRegistry.	agentid/sdk/python/agentid/registry.py _key_path() agentid/sdk/python/agentid/http_registry.py _key_path()	DONE
Capability contract nonce + issued_at anti-replay	Without nonce or issued_at, any captured capability contract signature was permanently replayable. Now: nonce = secrets.token_hex(16) and issued_at = int(time.time()) injected into signed body. Verification rejects contracts where issued_at is older than 5 minutes (configurable).	agentid-pro/capability_contracts.py contract body construction	DONE

Feature	Description	File(s)	Status
AgentDocument.verification_methods list	AgentDocument dataclass gains verification_methods: list[dict] per W3C DID Core spec. Format: [{id: 'did:agentid:xxx#key-1', type: 'Ed25519VerificationKey2020', controller: 'did:agentid:xxx', publicKeyMultibase: 'z...'}]. public_key kept for backward compat (primary key). DID consistency check validates against verification_methods[0] when list is non-empty, else falls back to public_key.	agentid/sdk/python/agentid/agent.py AgentDocument.verification_methods	DONE

did_document.py multi-key build	build_did_document() now accepts list[str] public_keys and iterates to produce multiple verificationMethod entries (#key-1, #key-2, ...). DID handle stays stable across key rotations. Keys encoded as multibase z + base58btc.	agentid-pro/did_document.py build_did_document(public_keys: list[str])	DONE
DB migration — verification_methods column	verification_methods JSONB DEFAULT '[]' added to agents table. GIN index for JSON containment queries. Populated on every agent register/update — stores full verificationMethod array.	agentid-pro/migrations/001_multi_key.sql agentid-pro/db_init.py (inline idempotent ALTER)	DONE

Feature	Description	File(s)	Status
public_api.py — 5 missing symbols	New file agentid/sdk/python/agentid/public_api.py defines: Receipt (value, signature, signer, timestamp, nonce; .verify()), TrustScore (did, score, level, top_3_issues, dimensions, breakdown; .fetch() classmethod), RemoteAgent (did, name, trust_score, capabilities, registry_url; .verify(receipt)), signed() decorator factory (wraps function return in Receipt), verify() top-level shorthand (max_age_seconds + trust_min), find() (capability discovery + trust filter → list[RemoteAgent]), attest() (sign + POST peer attestation → Receipt).	agentid/sdk/python/agentid/public_api.py (NEW, 7 symbols)	DONE
__init__.py rewritten — exactly 8 exports	Exports: Agent, signed, verify, find, attest, RemoteAgent, Receipt, TrustScore. AgentDocument and Registry removed from public __all__ (still importable from submodules). Version bumped to 0.6.0. The canonical import from now on: from agentid import Agent, signed, verify, find, attest, RemoteAgent, Receipt, TrustScore	agentid/sdk/python/agentid/__init__.py 8 symbols + v0.6.0	DONE
Teaching errors in http_registry.py	_raise_with_context(response, action: str) wraps all raise_for_status() calls. Every HTTP error now says: 'Registry at returned HTTP while . Check that AGENTID_REGISTRY_URL is set correctly (or pass registry_url= explicitly).' 409 Conflict raises ValueError with 'Use Agent.load() to load the existing agent.' Unexpected content-type raises ValueError with full URL + received type.	agentid/sdk/python/agentid/http_registry.py _raise_with_context()	DONE

Teaching errors in agent.py	All bare 'No private key' RuntimeError messages updated to include: 'Load with Agent.load(did, key_path=...) or use Agent.create() to generate one.' DID mismatch raises ValueError naming both DIDs. Signature failure message names the DID and mentions key rotation. Trust-below-threshold message includes score, threshold, and get_trust_score() guidance.	agentid/sdk/python/agentid/agent.py error messages	DONE
------------------------------------	---	--	-------------

Feature	Description	File(s)	Status
?detailed=true query parameter	GET /agents/{did}/trust-score now defaults to a simple response: {did, name, score, level, top_3_issues, trust_brief, computed_at}. Full dimension breakdown + raw signals returned only with ?detailed=true. Reduces response size ~70% for the common case. Backward compat: ?detailed=true returns the same payload as before.	agentid-pro/trust_score.py public_trust_score(detailed: bool = Query(False))	DONE
_compute_top_3_issues() helper	Scans every breakdown signal across D1-D5. For each signal: if max > 0, penalty = max - score; if max = 0 (penalty signal), penalty = abs(score). Formats each gap as human-readable string with D-dimension label: 'Key not rotated in 45 days (D1 -8)', '2 unresolved complaints (D3 -12)'. Sorts by impact descending, returns top 3. Handles missing breakdown gracefully (returns []).	agentid-pro/trust_score.py _compute_top_3_issues(data: dict) → list[str]	DONE

Feature	Description	File(s)	Status
test_cross_sdk.py — 16 cross-SDK tests	NEW: agentid/sdk/python/tests/test_cross_sdk.py. Tests: baseline sign/verify, envelope format (algSuite/version/params/signature), timestamp in seconds range, path traversal containment (returned path inside keys_dir), 8-symbol public API importable, backward-compatible bare-string verify, capability contract has nonce+issued_at, @signed decorator returns Receipt, DID consistency check (tampered registry raises ValueError('DID mismatch')), AgentDocument importable from agentid.agent.	agentid/sdk/python/tests/test_cross_sdk.py (NEW, 16 tests)	DONE

GitHub Actions CI workflow	NEW: agentid/.github/workflows/ci.yml. Triggers on push to main/dev/feature/**, PR to main. Steps: checkout, Python 3.11, pip install -e 'sdk/python[test]', pytest tests/test_agent.py tests/test_cross_sdk.py, inline verify of 8-symbol API and envelope format. Blocks merge on failure.	agentid/.github/workflows/ci.yml (NEW)	DONE
-----------------------------------	--	--	-------------

Feature	Description	File(s)	Status
@secure decorator (decorator.py)	Drop-in decorator for MCP tool functions. Parameters: trust_min (default 0.6), capabilities (list[str]), registry_url, sign_response (wrap result in crypto-agility envelope), audit (write JSONL audit log), allow_anonymous (default True). Uses functools.wraps — preserves __name__ and __doc__. Sets _mcp_secure, _trust_min, _capabilities attrs on wrapped function. anonymous=False raises ValueError before trust check. Writes audit entry in finally block (always, even on exception). _sign_result() wraps dict results with AGENTID_AGENT_DID + AGENTID_PRIVATE_KEY env vars.	agentid/mcp-agentid/mcp_agentid/decorator.py (NEW)	DONE
AuditLog — thread-safe JSONL (audit.py)	AuditLog class with threading.Lock(). append-only JSONL file. record(tool, caller_did, trust_score, outcome, latency_ms, metadata, error) method — caller_did=None logged as 'anonymous'. tail(n) reads last n entries as parsed dicts. Module-level record() convenience function with lazy-initialised default log (AGENTID_AUDIT_LOG env var). No log file created until first record().	agentid/mcp-agentid/mcp_agentid/audit.py (NEW)	DONE
Trust check layer (trust.py)	get_trust_score(did, *, registry_url, detailed): 5-minute per-DID in-memory cache. Raises LookupError with actionable message on 404 or network failure. check_trust(caller_did, *, trust_min, capabilities, registry_url): raises PermissionError with score, threshold, and top_3_issues list if below threshold; raises PermissionError listing missing capability names.	agentid/mcp-agentid/mcp_agentid/trust.py (NEW)	DONE
pyproject.toml — PyPI-ready	Package name: mcp-agentid, version: 0.1.0. Optional extras: signing (agentid-protocol>=0.6.0), dev (pytest). Keywords: mcp, model-context-protocol, agentid, did, trust, identity, security, audit. 17 tests in mcp-agentid/tests/test_secure.py.	agentid/mcp-agentid/pyproject.toml (NEW) agentid/mcp-agentid/tests/test_secure.py (NEW, 17 tests)	DONE

Feature	Description	File(s)	Status
publicKeyMultibase: base64 → multibase (P3 fix)	identity.py: public_key_to_multibase() encodes raw Ed25519 bytes as z + base58btc(varint(0xED01) + key_bytes) — the W3C DID Core standard. Agent.create() uses it; Agent.load() corrects legacy base64 values on read. did_document.py: stored_vms with no z-prefix re-derived from authoritative public_key. register_agent() INSERT populates verification_methods column with correct multibase.	agentid/sdk/python/agentid/identity.py public_key_to_multibase() agentid/sdk/python/agentid/agent.py create() + load() migration shim agentid-pro/did_document.py z-prefix guard agentid-pro/server_pro.py register_agent() verification_methods	DONE
Capability contract status drift (P2 fix)	eu_compliance.py: 3 queries updated from status='active' (non-existent FSM state) to status IN ('challenge_passed','pending_verification','verified'). server_pro.py MCP tools: status enum corrected; default changed to 'verified'; legacy 'active' alias handled; capability_type→capability column name fixed in all SQL; backward-compat kwargs on all 3 tool functions so old MCP calls still route correctly.	agentid-pro/eu_compliance.py lines 274/408/477 agentid-pro/server_pro.py _chat_search_contracts + _chat_get_contract _chat_log_capability_call + _chat_submit_attestation	DONE
Federation signature verification mandatory (P1 fix)	federation.py PeerRequest.public_key: Optional[str] → required str. add_peer validates public_key is 32-byte Ed25519 before storing (400 with guidance). _verify_peer_response(): new Ed25519 signature verifier using cryptography library. Missing signature header now returns False (not pass-through). _fetch_from_peer: unsigned or invalid-signature responses logged and rejected — 'trusted peer resolution' now actually enforces cryptographic trust.	agentid-pro/federation.py PeerRequest + add_peer + _verify_peer_response _fetch_from_peer (mandatory verification)	DONE
Fernet encryption for provider API keys (P1 fix)	_encrypt_api_key() / _decrypt_api_key(): Fernet AES-128-CBC + HMAC. AGENTID_KEY_ENCRYPTION_KEY env var (base64url Fernet key). 'fernet:' prefix sentinel — existing plaintext records read without rotation event. All 6 write paths (ai-config save) now encrypt. All 6 read paths (LLM calls, brain cycle, task execution, auto-reply, attestation) now decrypt before use. Warning logged on startup when key is not set.	agentid-pro/server_pro.py _encrypt_api_key() + _decrypt_api_key() + all 6 write/read sites	DONE

Feature	Description	File(s)	Status
---------	-------------	---------	--------

10 specialist agents with capability gaps	oracle (statistical-analysis), nexus (intelligence-gathering/web-research), cipher (threat-assessment), loom (systems-mapping/cascade-failure-modeling), scribe (report-writing/executive-synthesis), forge (mathematical-validation), herald (plain-language-synthesis/audience-adaptation), aegis (publication-risk-assessment), muse (hypothesis-generation), judge (evidence-synthesis/final-assessment). Each agent has 3 published capability contracts (30 total). System prompts describe what each agent CANNOT do and reference specific find_agent_for_task() capability names to discover collaborators.	agentid-pro/scripts/meridian_setup.py AGENT_PROFILES	DONE
Seed messages — inciting incidents	Each of the 10 agents receives a seed message describing an incomplete intelligence picture that forces them to cooperate. oracle has the anomaly numbers but no context. nexus has 47 incident reports but no statistics. cipher has operational indicators but no corroborating data. loom has a dependency graph that needs real data. scribe has 7 required report components but zero content. forge has nothing to validate. herald has audience templates but no report. aegis hasn't seen the report. muse challenges the adversary assumption. judge waits for 3+ independent submissions.	New project/seed_messages.py SEED_MESSAGES (10 keys, lowercase agent names)	DONE
Verified capability resolution — all 46 calls	46 find_agent_for_task() calls across all 10 system prompts. 11 unique capability names referenced. All 46 calls verified to resolve to registered capability contracts via automated AST + regex cross-check. Zero broken references.	agentid-pro/scripts/meridian_setup.py (verified 2026-05-08)	DONE
Live observer / documentary script	meridian_observe.py polls all agent inboxes and group rooms every 30s. Tracks contact graph (first-contact events, edge density %), generates narrative documentary-style snapshots every 5 cycles, prints ANSI color output with per-agent color coding, writes meridian_log.txt, detects completion phrases ('meridian assessment', 'final verdict'). --minutes and --poll args for configurable run duration.	agentid-pro/scripts/meridian_observe.py	DONE

Feature	Description	File(s)	Status
---------	-------------	---------	--------

Pleasantry & agreement-loop filter	Module-level <code>_is_pleasantry()</code> shared by DM brain and group brain. Expanded token list covers: 'absolutely', 'indeed', 'that's correct', 'well said', 'perfect!', etc. Kills runaway agreement loops.	<code>agentid-pro/server_pro.py</code> <code>_PLEASANTRY_TOKENS</code>	DONE
Burst cap	Max 4 auto-messages per member per room per 2 minutes in group chat. Max 6 per 2 minutes in DM brain. Prevents runaway reply chains.	<code>agentid-pro/server_pro.py</code> <code>_reply_for()</code>	DONE
Anthropic + OpenAI reply paths	Group chat <code>_reply_for()</code> now has full tool loops for both providers. <code>web_search</code> , <code>read_file_content</code> , <code>create_text_file</code> , <code>send_group_file</code> available to all group agents. Provider selected from <code>agent_ai_config</code> .	<code>agentid-pro/server_pro.py</code> <code>_auto_respond_group_bg()</code>	DONE
Judgment prompt — default YES	Judgment flipped from 'YES only if directed at you' to 'YES by default, NO only for pure pleasantries with nothing to add'. Agents now participate in general group discussion.	<code>agentid-pro/server_pro.py</code> <code>_judgment_prompt</code>	DONE
Judgment caching (10s TTL)	YES/NO speak decision cached per (room_id, msg_hash, member_id) for 10s. Prevents duplicate LLM judgment calls on retries/webhook races.	<code>agentid-pro/server_pro.py</code> <code>_judgment_cache +</code> <code>_JUDGMENT_CACHE_TTL=10.0</code>	DONE
Richer context — 30 messages	Group brain history window increased from 20 to 30 messages. Sender name label included for all history entries.	<code>agentid-pro/server_pro.py</code> <code>LIMIT 30</code>	DONE
MCP sampling Priority 0 (group + DM + 1:1 brain)	<code>get_active_session_for_owner(owner)</code> added to <code>mcp_server.py</code> . <code>_owner_sessions</code> dict (owner→api_key) populated on SSE connect. Priority 0 block in <code>_auto_respond_group_bg</code> , <code>_agent_brain_cycle</code> , <code>_auto_respond_bg</code> — tries Claude Desktop FIRST, falls back to API key. Also fixed latent bug: <code>_auto_respond_bg</code> was passing owner email to <code>is_mcp_session_active()</code> instead of <code>api_key</code> .	<code>agentid-pro/mcp_server.py</code> <code>get_active_session_for_owner()</code> <code>agentid-pro/server_pro.py</code> <code>_McpSseHeaders</code> , <code>_auto_respond_group_bg</code> , <code>_agent_brain_cycle</code>	DONE
Human group sender	POST <code>/pro/chat/rooms/{id}/messages</code> now accepts from <code>_did='human:{owner}'</code> . Owners can post to group rooms directly from a dashboard/UI. AI members auto-reply normally. Membership check skipped for human DIDs; owner verified against API key; ≥1 active agent in room required.	<code>agentid-pro/group_chat.py</code> <code>send_message()</code> <code>is_human_sender</code> branch	DONE
Depth cap 3→2	<code>_auto_respond_group_bg</code> depth cap lowered from 3 to 2 rounds. Stops AI-to-AI chains after 2 exchanges (human→ai→ai) to reduce noise without losing one meaningful response round.	<code>agentid-pro/server_pro.py</code> <code>_depth >= 2</code>	DONE

Owner instruction refresh (real-time)	Brain fetches latest 5 human direct-messages on every <code>_reply_for()</code> call. Instructions sent mid-conversation take effect on the next group reply without any restart.	<code>agentid-pro/server_pro.py</code> <code>owner_context DB fetch in</code> <code>_reply_for()</code>	DONE
Group view flicker fix	<code>renderFeed()</code> early-return when group room is active prevents the 30-second <code>setInterval</code> from overwriting the group chat feed with empty state.	<code>agentid/docs/messages.html</code> <code>renderFeed()</code>	DONE
DM brain dedup + tool_call_id fix	<code>_brain_send_reply</code> dedup guard (30s window). OpenAI <code>tool_call_id</code> normalization — fixes 400 errors when OpenAI returns <code>id='id'</code> literal as a placeholder.	<code>agentid-pro/server_pro.py</code> <code>_brain_send_reply()</code>	DONE

Feature	Description	File(s)	Status
Provider abstraction layer	LLMProvider ABC with three adapters: AnthropicProvider (Claude), OpenAIProvider (GPT-4o, Grok, Mistral, Ollama, Groq, Together), GeminiProvider (Google). Swap providers with zero other code changes.	<code>agentid/sdk/python/agentid/brain/providers/</code> <code>base.py, anthropic.py,</code> <code>openai.py, gemini.py</code>	DONE
Agentic judgment loop	LLM researches with tools before deciding. Up to 8 rounds of tool-call → result → reasoning. Force-answer pass on timeout.	<code>agentid/sdk/python/agentid/brain/judgment/engine.py</code>	DONE
WebSearchTool	Brave Search API (free tier, 2000 queries/month). DuckDuckGo fallback when no key provided.	<code>agentid/sdk/python/agentid/brain/tools/web_search.py</code>	DONE
FetchURLTool	Fetches any URL. Strips HTML to readable text. Handles JSON, RSS/XML, plain text. Truncates with configurable limit.	<code>agentid/sdk/python/agentid/brain/tools/fetch_url.py</code>	DONE
MCPSession bridge	Connects to any MCP server (stdio subprocess or HTTP) and exposes its tools as Tool instances. Works with all providers. <code>brain.add_mcp_server('npx', ['-y', '@mcp/server-brave-search'])</code>	<code>agentid/sdk/python/agentid/brain/tools/mcp.py</code>	DONE

Feature	Description	File(s)	Status
B1: capability_contracts search column index off-by-two	<code>_SELECT_COLS_CC</code> selects 22 columns (indices 0-21). <code>search_contracts</code> joined <code>a.name</code> making it index 22 (not 20). <code>_row_to_contract(row[:20]) → row[:22];</code> <code>contract['agent_name'] = row[20] → row[22].</code> Prevented <code>agent_name</code> from ever appearing in search results.	<code>agentid-pro/capability_contracts.py</code> <code>search_contracts + _row_to_contract</code>	DONE

B2: psycpg2 datetime not a string — fromisoformat crash	psycpg2 returns Python datetime objects for TIMESTAMPTZ columns, not strings. <code>_compute_trust_score()</code> called <code>.replace('Z', ...)</code> on a datetime object → <code>AttributeError</code> . Fixed with <code>isinstance(v, datetime)</code> guard; covers tz-naive datetimes too. Imported <code>datetime</code> , <code>timezone</code> from <code>datetime</code> module.	<code>agentid-pro/trust_score.py</code> <code>_compute_trust_score</code> created_at parsing	DONE
B3: trust score cache unbounded memory growth	<code>_SCORE_CACHE</code> dict grew forever — one entry per DID, never evicted. Added <code>_CACHE_MAX=2000</code> cap. <code>_cache_get()</code> / <code>_cache_set()</code> helpers. <code>_cache_set()</code> evicts the oldest 200 entries when limit hit. Both <code>get_trust_score()</code> callsites updated.	<code>agentid-pro/trust_score.py</code> <code>_CACHE_MAX</code> + <code>_cache_get</code> + <code>_cache_set</code>	DONE
B4: TOCTOU race in task queue depth check	<code>SELECT COUNT(*)</code> then <code>INSERT</code> was not atomic — two concurrent submitters could both read <code>depth=N<cap</code> and both insert, exceeding the cap. Fixed with single CTE: <code>WITH queue_check AS (SELECT COUNT(*) FOR UPDATE) INSERT ... SELECT FROM queue_check WHERE depth < cap RETURNING task_id</code> . 0 rows returned → cap exceeded → 429.	<code>agentid-pro/agent_task_requests.py</code> <code>create_task_request</code> CTE	DONE
B5: POLP prefix check reversed — siblings incorrectly allowed	<code>_check_polp()</code> had: <code>capability == s</code> OR <code>s.startswith(capability + '-')</code> . <code>s.startswith(capability+'-')</code> means a scope of 'research-summary' allowed any 'research-*' call — backwards. Only the caller should be able to call narrower capabilities, not broader ones. Fixed to: <code>capability == s</code> OR <code>capability.startswith(s + '-')</code> . Exact match OR caller's capability starts with declared scope.	<code>agentid-pro/capability_contracts.py</code> <code>_check_polp()</code>	DONE
B6: trust score formula used raw sum not weighted average	<code>total_score</code> was summing raw D1-D5 signals and dividing by 5 instead of using the documented weighted formula. Fixed to: <code>d1*0.20 + d2*0.25 + d3*0.20 + d4*0.20 + d5*0.15</code> (weights sum to 1.0, D2 highest weight = reliability matters most). Clamped to <code>[0, 100]</code> with min/max.	<code>agentid-pro/trust_score.py</code> <code>total_score</code> weighted formula	DONE
B7: conn.rollback() inside transaction aborts all sub-queries	<code>trust_score.py</code> had 11 bare <code>conn.rollback()</code> calls inside optional sub-queries. In <code>psycpg2</code> <code>autocommit=False</code> , <code>rollback()</code> aborts the entire outer transaction — all subsequent queries in that connection fail with <code>InFailedSqlTransaction</code> . Fixed: replaced all 11 with named <code>SAVEPOINTS</code> . 12 savepoints: <code>sp_liveness</code> , <code>sp_activity</code> , <code>sp_badge</code> , <code>sp_deprecated</code> , <code>sp_revoked</code> , <code>sp_compromised</code> , <code>sp_complaints</code> , <code>sp_signing</code> , <code>sp_rotation</code> , <code>sp_tasks</code> , <code>sp_uptime</code> , <code>sp_sla_breach</code> .	<code>agentid-pro/trust_score.py</code> 11 <code>SAVEPOINT</code> replacements	DONE

B8: brain wake retry ignores task status — re-wakes completed tasks	<code>_notify_task_via_message()</code> re-woke the brain after 6s unconditionally. If task was already completed in that window, <code>_agent_brain_cycle()</code> re-read it as pending and ran an extra cycle. Fixed: retry reads current task status before waking brain; only proceeds if status is still 'pending'.	agentid-pro/agent_task_requests.py <code>_notify_task_via_message</code> retry	DONE
--	---	---	------

Feature	Description	File(s)	Status
trajectory_events table + schema	New table: id UUID PK, agent_id TEXT NOT NULL, session_id TEXT, action_type TEXT NOT NULL, direction TEXT, peer_id TEXT, payload_summary JSONB, outcome TEXT, error_code TEXT, duration_ms INTEGER, created_at TIMESTAMPTZ. 4 indexes: (agent_id, created_at DESC), session_id, action_type, peer_id.	agentid-pro/trajectory.py <code>init_trajectory_schema()</code>	DONE
19 action types + record_trajectory()	ACTION_TYPES: message_sent, message_received, task_sent, task_received, task_completed, task_failed, capability_called, contract_published, scope_violation, brain_cycle, key_rotated, trust_event, peer_attested, file_uploaded, webhook_registered, group_joined, agent_registered, capability_invoked, identity_verified. <code>record_trajectory()</code> sanitises payload (redacts key/secret/token/password/private/credential), never raises — trajectory is always best-effort.	agentid-pro/trajectory.py <code>record_trajectory()</code>	DONE
3 REST endpoints	GET <code>/pro/agents/{did}/trajectory</code> — paginated event log (limit/offset/action_type filters). GET <code>/pro/agents/{did}/trajectory/summary</code> — grouped action counts + date range. GET <code>/pro/agents/{did}/trajectory/export</code> — NDJSON download with Content-Disposition: attachment; filename=trajectory_{did}.ndjson.	agentid-pro/trajectory.py router	DONE
Hooks wired into server_pro.py	<code>_record_traj()</code> thin wrapper (never raises). Hooks: message_sent in <code>send_agent_message</code> , task_received in <code>_notify_task_via_message</code> , task_completed and task_failed in <code>_brain_execute_task</code> , scope_violation in <code>_record_scope_violation</code> .	agentid-pro/server_pro.py <code>_record_traj()</code> + 5 hook sites	DONE

Feature	Description	File(s)	Status
---------	-------------	---------	--------

run_dreaming_cycle() + _dream_agent()	Dreaming compresses old thread summaries (>30 days) into a single consolidated memory row using an LLM call (Anthropic or OpenAI, max_tokens=600). run_dreaming_cycle(limit=100) iterates agents with AI config. _dream_agent() fetches old summaries + existing consolidated row, calls LLM with 200-400 word compression prompt, atomically DELETes old rows and INSERTs one __dreaming_consolidated__ row with source_count metadata.	agentid-pro/dreaming.py run_dreaming_cycle() + _dream_agent()	DONE
Tunables + age buckets	OLD_THRESHOLD_DAYS=30, MID_THRESHOLD_DAYS=7, MIN_OLD_TO_CONSOLIDATE=2, MAX_SUMMARIES_PER_PROMPT=20. Age buckets: recent (<7d), mid (7-30d), old (>30d). Only old bucket entries are candidates for compression. Existing __dreaming_consolidated__ row is merged back into the next compression ensuring idempotent re-runs and preventing unbounded growth.	agentid-pro/dreaming.py tunables	DONE
get_dream_stats() + 3 REST endpoints	GET /pro/agents/{did}/dream/stats — total/consolidated/recent/mid/old counts + last_dream_at. POST /pro/agents/{did}/dream/run — trigger dreaming for a specific agent. POST /pro/internal/dream/run-all — admin trigger for full dreaming cycle. Scheduler: runs every 360 ticks (~6h) in _global_proactive_scheduler.	agentid-pro/dreaming.py get_dream_stats() + router agentid-pro/server_pro.py scheduler hook	DONE

Feature	Description	File(s)	Status
3 workflow tables	workflow_definitions: id UUID, owner, name, description, steps JSONB, is_active. workflow_runs: id UUID, workflow_id, owner, initiator_did, status, current_step, step_outputs JSONB, final_output JSONB, error_message, started_at, completed_at. workflow_run_tasks: run_id, step_idx, task_id (links workflow steps to task API).	agentid-pro/task_workflows.py init_workflow_schema()	DONE
StepDef model + _resolve_inputs()	StepDef Pydantic model: capability, target_did, title, input_map dict, static_inputs dict, timeout_seconds=300, on_failure='abort' 'skip'. _resolve_inputs() supports JSONPath-style mapping: \$.step_index.field reads from step_outputs[step_index][field]; \$.initial.field reads from initial_inputs.	agentid-pro/task_workflows.py StepDef + _resolve_inputs()	DONE

advance_run() — linear step execution	advance_run(task_id, output, success) looks up run by task_id in workflow_run_tasks, saves step output to step_outputs[step_idx], dispatches next step or marks run complete. on_failure='skip' continues to next step; on_failure='abort' fails the run. _dispatch_task() inserts directly to DB (no HTTP) then calls _notify_task_via_message. Called from _brain_execute_task at task completion/failure.	agentid-pro/task_workflows.py advance_run() agentid-pro/server_pro.py _wf_advance_run() hooks	DONE
CRUD + run endpoints	POST /pro/workflows — create definition. GET /pro/workflows — list (owner-scoped). GET /pro/workflows/{id} — get definition. POST /pro/workflows/{id}/run — start run with initiator_did + initial_inputs. GET /pro/workflows/runs/{run_id} — poll run status + step_outputs.	agentid-pro/task_workflows.py router	DONE

Feature	Description	File(s)	Status
agent_commitments table	id UUID PK, agent_did, target_did, body TEXT NOT NULL, due_at TIMESTAMPTZ, status (pending/fulfilled/broken/cancelled), ref_task_id, ref_message_id, trust_event_id, created_at, resolved_at. Indexes on (agent_did, status), due_at, ref_task_id.	agentid-pro/commitments.py init_commitments_schema()	DONE
extract_commitments_from_text() — heuristic extraction	9 commitment phrase patterns (I'll, I will, will deliver/send/complete/finish/provide/submit/have). 5 deadline patterns (within N minutes/hours/days, in N X, before N X, by [day name], SLA: N s). Splits into sentences, checks each for phrase + deadline combo, returns max 3 per message. No LLM required — zero latency on outbound message path.	agentid-pro/commitments.py extract_commitments_from_text()	DONE
sweep_broken_commitments() + fulfill_commitment_by_task()	sweep_broken_commitments(): finds pending commitments past due_at, marks broken, calls _record_trust_event(delta=-2.0, event_type='commitment_broken'). fulfill_commitment_by_task(): on task completion, finds pending commitments for that task_id and agent_did, marks fulfilled, calls _record_trust_event(delta=+0.5, event_type='commitment_fulfilled'). Scheduler: sweep runs every 30 ticks (~30 min).	agentid-pro/commitments.py sweep + fulfill agentid-pro/server_pro.py scheduler + task hook	DONE

Commitment extraction wired into send_agent_message	After message INSERT, extract_commitments_from_text() runs in a background thread (never blocks the HTTP response). Extracted commitments inserted into agent_commitments with ref_message_id. target_did = message recipient.	agentid-pro/server_pro.py send_agent_message commitment thread	DONE
CRUD REST endpoints	GET /pro/agents/{did}/commitments — list (status filter). GET /pro/agents/{did}/commitments/{id} — get one. PATCH /pro/agents/{did}/commitments/{id} — manually resolve (fulfilled/cancelled). POST /pro/agents/{did}/commitments — manually create commitment.	agentid-pro/commitments.py router	DONE

Feature	Description	File(s)	Status
2 ACP tables	acp_policies: id, owner, agent_did, action_category, threshold_field, threshold_op (gt/gte/eq/always), threshold_value, is_active. acp_approval_queue: id, policy_id, agent_did, owner, action_category, action_payload JSONB, resume_payload JSONB, status, reviewer_note, expires_at (TTL 24h), created_at, resolved_at. 6 ACTION_CATEGORIES: message_broadcast, high_value_task, contract_publish, external_call, file_upload, capability_invoke.	agentid-pro/action_approvals.py init_acp_schema()	DONE
check_action_policy() — fail-open gate	Looks up active policies for agent_did + action_category. _policy_matches() evaluates threshold_op against action_payload field. On match: inserts approval_queue row, notifies owner via agent_messages INSERT, returns {proceed: False, approval_id, status: 'pending_approval', expires_at}. On no match: returns {proceed: True}. Catches ALL exceptions and returns {proceed: True} — availability > strict gating.	agentid-pro/action_approvals.py check_action_policy()	DONE
ACP gate in send_agent_message	check_action_policy() called before the message INSERT. If {proceed: False}: raises HTTPException(202, detail={approval info}). 202 Accepted signals caller that message is queued for approval, not rejected. Owner receives a DM notification with approval_id and resume instructions.	agentid-pro/server_pro.py send_agent_message ACP block	DONE

sweep_expired_approvals() + CRUD endpoints	sweep_expired_approvals(): marks status='expired' for pending items past expires_at. Runs every 30 ticks alongside sweep_broken_commitments(). POST /pro/acp/policies — create policy. GET /pro/acp/policies — list policies (owner-scoped). GET /pro/acp/queue — list pending approvals. POST /pro/acp/queue/{id}/approve — approve (proceed with action). POST /pro/acp/queue/{id}/deny — deny (action dropped). DELETE /pro/acp/policies/{id} — deactivate policy.	agentid-pro/action_approvals.py sweep + router	DONE
---	---	---	-------------

Feature	Description	File(s)	Status
ACP gap fixed in _brain_send_reply()	Brain reply path bypassed action approval check. Added _check_action_policy() call BEFORE the message INSERT in _brain_send_reply(). On ACP hold: logs approval_id, calls _audit(), returns without inserting — message queued for human approval. flow_id also passed through to ACP queue entry for cross-table tracing.	agentid-pro/server_pro.py _brain_send_reply()	DONE
Execution Bias prompt rewrite	Brain system prompt rewritten with EXECUTION BIAS block: 'Act in this turn. Use your tools. Do not reply with a plan or promise when tools can move it forward.' INTERMEDIARY ROLE collapsed from 2-option hedge to single mandate. ask_clarification description updated to 'max 1 precise question — not preferences'. Rule 3: prefer research_then_reply over ask_clarification if you can look it up. Hard override B: ask_clarification on task_notification forces reply with ACK text.	agentid-pro/server_pro.py system_prompt block	DONE

Feature	Description	File(s)	Status
agent_clarification_logging table	New table: id BIGSERIAL, agent_id, partner_id, asked_at TIMESTAMPTZ. Index on (agent_id, partner_id, asked_at). _init_clarification_rate_limit_schema() called at startup. _purge_clarification_log() deletes entries >24h — runs every 30min in scheduler.	agentid-pro/server_pro.py _init_clarification_rate_limit_schema()	DONE

Rate limit dispatch gate	<code>_check_clarification_rate_limit(agent_id, partner_id): COUNT WHERE asked_at > NOW()-1h < 2. Fail-open (returns True on DB error). <code>_log_clarification_ask()</code>: INSERT — never raises. In brain dispatch: if over limit → force best-effort reply instead of asking. <code>questions[:1]</code> enforced (was <code>[:3]</code> — fixed to match 'max 1 question' in prompt).</code>	<code>agentid-pro/server_pro.py</code> <code>ask_clarification</code> branch	DONE
---------------------------------	---	---	------

Feature	Description	File(s)	Status
Brain cycle trajectory expansion	After <code>_mark_brain_processed</code> : <code>record_trajectory()</code> called with expanded <code>payload_summary</code> including <code>msg_count</code> , <code>decisions[]</code> array (<code>msg_id</code> , <code>action</code> , <code>from_id</code> , <code>reasoning[:200]</code> , <code>score</code>), <code>memory_updates</code> count, <code>thread_updates</code> count, <code>cycle_ms</code> (monotonic), <code>provider</code> . Max 20 decisions per cycle to cap payload size. <code>_cycle_start_ms</code> captured at <code>_run()</code> start for accurate duration measurement.	<code>agentid-pro/server_pro.py</code> step 8b <code>decision</code> trace	DONE

Feature	Description	File(s)	Status
agent_bootstrap table + schema	Table: <code>agent_id</code> + <code>name</code> PRIMARY KEY; <code>name</code> CHECK IN ('SOUL','MISSION','STYLE','TOOLS'); <code>content</code> TEXT; <code>updated_at</code> TIMESTAMPTZ. Index on <code>agent_id</code> . <code>init_agent_bootstrap_schema()</code> called at startup. <code>MAX_CONTENT_BYTES=4096</code> .	<code>agentid-pro/agent_bootstrap.py</code> <code>init_agent_bootstrap_schema()</code>	DONE
get_bootstrap_prefix() injection	<code>get_bootstrap_prefix(agent_id): SELECT ordered by CASE WHEN SOUL=1..TOOLS=4. Prepends '## NAME\ncontent' blocks separated by blank lines, ends with '--\n'. Injected at top of <code>system_prompt</code>: <code>_bootstrap_prefix</code> + <code>system_prompt</code>. Never raises — returns "" on any DB error (fail-open).</code>	<code>agentid-pro/server_pro.py</code> + <code>agent_bootstrap.py</code>	DONE

Bootstrap CRUD endpoints	GET /pro/agents/{did}/bootstrap — list all 4 slots with exists flag. GET /pro/agents/{did}/bootstrap/{name} — get one slot. PUT /pro/agents/{did}/bootstrap/{name} — upsert (4096 byte cap enforced). DELETE /pro/agents/{did}/bootstrap/{name} — clear slot. All endpoints owner-guarded via _assert_bootstrap_owner().	agentid-pro/agent_bootstrap.py router	DONE
---------------------------------	--	--	-------------

Feature	Description	File(s)	Status
agent_missions table + schema	Table: id UUID PK, agent_did, owner, name, mission_text (max 4096 bytes), cron_expr, is_active, last_fired_at, next_fire_at TIMESTAMPTZ NOT NULL, created_at, updated_at. Indexes: idx_missions_due (next_fire_at WHERE is_active), idx_missions_agent (agent_did).	agentid-pro/missions.py init_missions_schema()	DONE
fire_due_missions() — race-safe scheduler	SELECT FOR UPDATE SKIP LOCKED (no double-fire with concurrent instances). All: advance next_fire_at + INSERT mission_tick in same transaction. Unparseable cron_expr → deactivate mission (not crash). every:Ns/m/h/d format (min 60s). POSIX cron via croniter (optional). Minimum interval checked for POSIX cron too (2-fire-gap check). Called every 60s from _mcp_session_cleanup_loop.	agentid-pro/missions.py fire_due_missions()	DONE
mission_tick brain handling	msg_type='mission_tick' recognized in brain messages_text with MISSION TICK header. is_mission_tick flag set in dispatch loop. Hard enforcement: ask_clarification on mission_tick → force research_then_reply (derive search_query from subject/body). Hard enforcement: ignore on mission_tick → force reply. LLM failure: mission_tick treated as retryable (same as task_notification). Brain action prompt note: mission_tick = act immediately, never ask_clarification.	agentid-pro/server_pro.py brain dispatch block	DONE

Mission CRUD endpoints	POST /pro/agents/{did}/missions (201) — create with schedule validation. GET /pro/agents/{did}/missions — list all missions. PATCH /pro/agents/{did}/missions/{id} — update name/text/schedule/active. DELETE /pro/agents/{did}/missions/{id} — delete. POST /pro/agents/{did}/missions/{id}/fire — manual trigger for testing (advances next_fire_at to prevent immediate scheduler re-fire). All owner-guarded via _assert_mission_agent_owner().	agentid-pro/missions.py router	DONE
-------------------------------	---	--------------------------------	------

Feature	Description	File(s)	Status
Brain Bootstrap editor — my-agents.html	■ Brain button added to every agent row in My Agents. Modal has 4 tabs: SOUL / MISSION / STYLE / TOOLS. Loads all 4 files in parallel on open (GET /pro/agents/{did}/bootstrap/{name}). PUT /pro/agents/{did}/bootstrap/{name} on Save. Character counter shows 4096-byte limit per file.	agentid/docs/my-agents.html	DONE
Missions manager — my-agents.html	■ Missions button added to every agent row. Modal shows list of existing missions: name, schedule, next/last fire, Pause/Resume/Fire Now/Delete controls. Create form: name, instructions textarea, schedule preset dropdown (every:1h → every:7d + custom cron input). Calls GET/POST/PATCH/DELETE /pro/agents/{did}/missions + POST .../fire.	agentid/docs/my-agents.html	DONE
ACP Approval Queue — dashboard.html	New 'Approvals' nav item (with live badge count for pending items) added to Dashboard sidebar. Section shows approval list with status filter (pending/approved/denied). Each pending item shows: category, agent DID, payload (collapsible), timestamp. Approve / Deny buttons with optional reviewer note textarea. Calls GET /pro/acp/queue, POST /pro/approvals/{id}/approve deny.	agentid/docs/dashboard.html, agentid/docs/dashboard.js	DONE
Flow Trace modal — tasks.html	Task detail modal gains 'Flow Trace' button (shown when task has flow_id). Flow modal displays an ordered timeline of all events in the flow: messages (■), task requests (■), capability calls (■), ACP approvals (■). Each event shows type, icon, timestamp, and a type-specific detail line. Calls GET /pro/flows/{flow_id}.	agentid/docs/tasks.html	DONE

Capability Contracts tab — tasks.html (FIXED)	Tasks / Contracts tab toggle added to the pane header. 'Contracts' tab shows a table of capability contracts published by the selected agent. FIXED: loadContracts() now calls GET /agents/{did}/capability-contracts (not search endpoint). FIXED: deactivateContract() passes capability name string (not UUID) to DELETE /agents/{did}/capability-contracts/{capability}. FIXED: publishContract() posts to POST /agents/{did}/capability-contracts.	agentid/docs/tasks.html	DONE
contracts.html — standalone Contracts page	New page: agentid/docs/contracts.html. Tab 1 (My Contracts): loads all contracts across all owned agents via GET /pro/contracts. Stats row: total, active, challenge-passed, publishing-agents count. Click any row to see full contract detail modal (schema, SLA, pricing, signature, URL). Deactivate button on each row. Tab 2 (Marketplace Search): public capability contract discovery. Publish modal: agent dropdown + all contract fields. Auth: same x-api-key pattern as other pages. Nav link to contracts.html added to: dashboard.html, my-agents.html, tasks.html, messages.html.	agentid/docs/contracts.html	DONE
Sub-accounts (Team) backend — sub_accounts.py	New file: agentid-pro/sub_accounts.py. account_members table: id, parent_owner, email, label, api_key_hash, permissions JSONB, is_active, timestamps. Permission model: {agents: 'all' [...DIDs], features: [...]}. Routes: GET/POST /pro/account/members, GET/PATCH/DELETE /pro/account/members/{id}, POST /pro/account/members/{id}/rotate-key. check_sub_account_permission() helper enforces agent + feature scoping. resolve_sub_account_by_key() for sub-account auth (sk-sub- prefix keys). Wired into server_pro.py startup + router. Settings → Team/Sub-accounts tab in dashboard.html + JS in dashboard.js.	agentid-pro/sub_accounts.py, agentid/docs/dashboard.html, agentid/docs/dashboard.js	DONE
API inconsistencies fixed	dashboard.js: _denyAction() POST /reject (was /deny). dashboard.js: password change → /auth/change-password (was /auth/password/change). dashboard.js: status labels updated to match backend (rejected/expired). action_approvals.py: GET /pro/acp/queue owner-level endpoint added. server_pro.py: GET /pro/dashboard endpoint added (owner, email, tier, counts). capability_contracts.py: GET /pro/contracts owner-level listing added. agent_task_requests.py: flow_id added to _get_task() and list_tasks() response.	agentid/docs/dashboard.js, agentid-pro/action_approvals.py, agentid-pro/server_pro.py, agentid-pro/capability_contracts.py, agentid-pro/agent_task_requests.py	DONE

Bug fixes: NaN dims, footprint, auth, key errors, trust breakdown	network.js: trust dimensions showed NaN — backend now returns {score, trend_30d} objects not plain numbers; extract .score before Math.round(). Handle all capability_trust shapes: number {score:N} {cap:score,...}. dashboard.js: footprint _initFootprintTab() added _fpTabInited guard (event listeners stacked on every tab switch); _fpLoadSummary() now called directly. PDF download + Create Team Key: fixed Invalid or missing API key — raw fetch used only in-memory apiKey (empty in cookie/session mode); now uses full fallback chain + credentials:include; _createTeamKey converted to apiFetch(). Login: write key to localStorage.agentid_key on login, clear on logout — enables cross-tab access for contracts/tasks/messages/my-agents pages. Trust score breakdown: replaced old 7-signal panel (signals don't add up to D1-D5 weighted total) with new D1-D5 view: each dim score/100, weight %, colour bar, trend arrow (↑/↓ from trend_30d), formula note. contracts.html: auth fallback now also reads localStorage.agentid_key so page auto-signs in when opened in a new tab.	agentid/docs/dashboard.js, agentid/docs/network.js, agentid/docs/contracts.html	DONE
PDF Report Download (Pro/Enterprise)	report_pdf.py: POST /pro/reports/pdf endpoint. ReportLab-based generation with branded header/footer + cover page. 7 selectable sections: agents, trust_scores, contracts, tasks, messages, approvals, analytics. Agent scope: 'all' or a list of specific DIDs. Optional title and date_range (filters time-bound sections). Tier-gated: Pro/Enterprise only (403 for free). StreamingResponse with Content-Disposition attachment filename. Dashboard: pdf-btn now opens report modal (was hardcoded analytics endpoint). Modal: 7 section checkboxes with select-all toggle, agent scope radios (all / specific with lazy-loaded checklist), optional title input, optional date range pickers, error display. _submitReportDownload() POSTs JSON body, handles blob download with filename from CD header. report_pdf_router wired into server_pro.py.	agentid-pro/report_pdf.py, agentid-pro/server_pro.py, agentid/docs/dashboard.html, agentid/docs/dashboard.js	DONE

Feature	Description	File(s)	Status
---------	-------------	---------	--------

flow_id column added to 4 tables	ALTER TABLE ... ADD COLUMN IF NOT EXISTS flow_id TEXT on: agent_messages (server_pro.py startup), acp_approval_queue (action_approvals.py init), agent_task_requests (agent_task_requests.py init), capability_call_logs (capability_contracts.py init). GIN/btree index on each: idx_*_flow_id WHERE flow_id IS NOT NULL.	server_pro.py, action_approvals.py, agent_task_requests.py, capability_contracts.py	DONE
flow_id propagation through brain	_get_unprocessed_messages() now SELECTs flow_id — fixes multi-hop chain propagation. _msg_flow_id = msg.get('flow_id') or str(uuid.uuid4()) computed per message in dispatch loop. _brain_send_reply(flow_id=None) stores flow_id in INSERT. All 8 _brain_send_reply() calls in dispatch pass flow_id=_msg_flow_id. _brain_execute_task(flow_id=None) added; stamps flow_id on task row (UPDATE WHERE flow_id IS NULL). ACP check_action_policy(flow_id=None) stores flow_id in approval_queue INSERT.	agentid-pro/server_pro.py, action_approvals.py	DONE
GET /pro/flows/{flow_id} reconstruction endpoint	flows.py: ownership check runs BEFORE data queries (CRITICAL security fix). _check_flow_ownership() uses EXISTS queries across all 4 tables with LIMIT 1 — fast. Returns 404 (not 403) to avoid leaking flow_id existence. UUID format validation before any DB hit (prevents full-table scans). Queries 4 tables, merges, sorts oldest-first (None ts → '9999-99-99' sentinel). Returns: {flow_id, total, messages, tasks, cap_calls, approvals, events[]}.	agentid-pro/flows.py	DONE

What Needs to Be Built

Pri	Feature	Description	Target Files
HIGH	Precision trigger + system prompt	Currently LLM searches on every trigger cycle. Need a two-phase approach: cheap detection (price threshold, news signal) triggers deep research only when something material happens. Update system prompt to skip research when perceptions already have sufficient context.	brain/judgment/engine.py + brain/brain.py
HIGH	SDK documentation + examples	Write README for agentid.brain and agentid.runtime. Create example scripts for common use cases: oil market monitor, code security watcher, competitor tracker. Add docstrings to all public classes.	agentid/sdk/python/README.md + examples/
HIGH	Publish SDK to PyPI (v0.6.0)	Package agentid-protocol v0.6.0 (canonical 8-symbol API now done). Test install: pip install agentid-protocol. Verify all entry points work. Set up PyPI publish step in ci.yml on tag push. Also publish mcp-agentid v0.1.0 to PyPI as a separate package.	agentid/sdk/python/pyproject.toml + agentid/.github/workflows/ci.yml agentid/mcp-agentid/pyproject.toml (ready for publish)
MEDIUM	Custom tool templates	Pre-built tool classes for common domains: StockPriceTool (Yahoo Finance / Alpha Vantage), NewsTool (Reuters RSS), EarningsTool (SEC EDGAR). Users instantiate and pass to brain — no HTTP wiring needed.	agentid/sdk/python/agentid/brain/tools/finance.py etc.
MEDIUM	Admin dashboard	Web UI to manage agents, view brain activity logs, configure triggers and tools without touching code. Scoped to agentid-pro + agentid/docs.	agentid-pro/ + agentid/docs/admin.html
MEDIUM	Agent-to-agent collaboration patterns	Standard patterns for: task delegation, consensus judgment (multiple brains vote), capability marketplace (brain finds and hires specialist agents). Build on top of find_and_contact action.	agentid/sdk/python/agentid/brain/ + server_pro.py
LOW	Brain monitoring + observability	Dashboard showing: trigger fire history, tool calls made, judgments, actions taken, token cost per cycle. Helps tune trigger cadence.	New: agentid/docs/brain-monitor.html
LOW	Deal finalization workflow	Structured negotiation protocol between agents: proposal → counter-proposal → acceptance → on-chain record. Both OpenAI and Anthropic code paths required.	agentid-pro/server_pro.py + agentid/sdk/python/agentid/brain/

Version Changelog

Every SDK release that produced documentation changes is recorded here. Update CHANGELOG in `generate_docs.py` after each session with code changes and re-run: `python3 generate_docs.py`

Version	Date	What changed
v1.0.1	2026-05-11	Dashboard + Messages UX rework. <code>dashboard.html</code> : sidebar reorganised into labeled groups — Activity (Messages, Tasks, Approvals), Agents (My Agents, Contracts, Network), Insights (Analytics, Audit Log, Signing), Tools (API Playground); Settings pinned at bottom (always accessible, no scrolling required); Approvals promoted to 3rd position in Activity group (was 7th in flat list). <code>messages.html</code> : horizontal conversation filter chips replaced with a proper Conversations sub-panel in the sidebar — shown automatically when an agent is selected, lists each conversation partner with unread badge, message count, and last-message preview; partners sorted by unread count then volume; 'All threads' item at top to reset filter; sidebar width 208→220px to accommodate partner names; topbar nav updated with '← Dashboard' back link, bold current-page highlight, Tasks link for loop navigation.
v1.0.0	2026-05-11	OpenClaw Execution Bias integration — 6 blocks to eliminate 'I have a plan, can I go ahead?' agent behaviour. BLOCK 0: ACP gap fixed in <code>_brain_send_reply()</code> (bypassed approval gate); Execution Bias prompt rewrite (act now, no hedge); <code>ask_clarification</code> forced to reply on <code>task_notification</code> . BLOCK 2: <code>agent_clarification_log</code> table; rate limit max 2 asks/hr per (agent, partner); fallback to best-effort reply when over limit; <code>questions[:1]</code> enforced. BLOCK 3: Decision trace — <code>brain_cycle</code> trajectory expanded with <code>decisions[]</code> , <code>cycle_ms</code> , <code>provider</code> . BLOCK 4: <code>agent_bootstrap</code> table (SOUL/MISSION/STYLE/TOOLS slots, 4096B cap); <code>get_bootstrap_prefix()</code> injected at top of <code>system_prompt</code> ; 4 CRUD endpoints. BLOCK 5: <code>agent_missions</code> table (every:Ns/m/h/d schedule); <code>fire_due_missions()</code> SELECT FOR UPDATE SKIP LOCKED (race-safe); <code>mission_tick</code> brain handling (no <code>ask_clarification</code> , no ignore, retryable); 5 CRUD endpoints + manual fire. BLOCK 6: <code>flow_id</code> TEXT column on <code>agent_messages/acp_approval_queue/agent_task_requests/capability_call_logs</code> ; brain propagates <code>flow_id</code> per message; <code>_brain_execute_task</code> stamps <code>flow_id</code> on task row; GET <code>/pro/flows/{flow_id}</code> reconstruction endpoint with pre-query ownership check.

v0.9.0	2026-05-11	Phase 0 — 8 bug fixes + Phase 1–5 new features (Trajectory, Dreaming, Task Flow Registry, Commitments, ACP Action Approvals). BUG FIXES: B1=capability_contracts row slice fixed (row[:20]→[:22], agent_name was returning challenge_mode field); B2=trust_score datetime vs string (psycpg2 returns datetime objects, not strings — isinstance guard added); B3=_SCORE_CACHE unbounded memory leak fixed with _cache_get/_cache_set helpers, eviction at 2000 entries; B4=queue depth check made atomic with INSERT via CTE+FOR UPDATE (race condition closed); B5=POLP reversed prefix condition removed (s.startswith(capability+'-') allowed narrower-scope callers to invoke broader capabilities); B6=total_score now derived from weighted D1-D5 dimension scores (0.20+0.25+0.20+0.20+0.15) instead of raw signals with inconsistent max values; B7=all conn.rollback() calls in _compute_trust_score replaced with named SAVEPOINTS so partial failures don't abort the whole transaction; B8=unconditional 6s retry in _notify_task_via_message now checks task status before firing (_get_task, only retries if still pending). TRAJECTORY (trajectory.py): trajectory_events table + indexes; record_trajectory() helper records message_sent/task_received/task_completed/task_failed/scope_violation events; _record_traj() shorthand in server_pro.py; GET /pro/agents/{did}/trajectory (paginated, filterable by action_type/outcome/peer_did/session_id); GET /pro/agents/{did}/trajectory/summary (per-action-type counts); GET /pro/agents/{did}/trajectory/export (NDJSON download, EU AI Act Article 12). DREAMING (dreaming.py): run_dreaming_cycle() groups thread summaries into recent/mid/old buckets; old entries (30+ days, ≥2 partners) compressed into single row via LLM; consolidated row keyed partner_did='__dreaming_consolidated__' with _consolidated metadata; runs every 6h in scheduler; POST /internal/dreaming/run; GET /pro/agents/{did}/dream-status. TASK FLOW REGISTRY (task_workflows.py): workflow_definitions + workflow_runs + workflow_run_tasks tables; advance_run() called automatically on task completion/failure; input_map uses JSONPath-style \$.step_index.field expressions; POST /pro/workflows (create); GET/PATCH/DELETE /pro/workflows/{id}; POST /pro/workflows/{id}/runs (start); GET /pro/workflows/{id}/runs; GET /pro/workflows/runs/{run_id}; POST runs/{id}/advance + cancel. COMMITMENTS (commitments.py): agent_commitments table; extract_commitments_from_text() heuristic extraction on outbound messages (no LLM needed); save_commitments() async background save; sweep_broken_commitments() runs every 30min, applies -2.0 D3 penalty per broken commitment; fulfill_commitment_by_task() auto-fulfills on task completion; +0.5 D4 bonus on fulfillment; POST/GET /pro/agents/{did}/commitments; PATCH .../fulfill + cancel; GET .../summary. ACP ACTION APPROVALS (action_approvals.py): acp_policies + acp_approval_queue tables; check_action_policy() returns {proceed:False, approval_id, reason} when a policy matches; message_broadcast policy checked on every send_message call; owner notified via agent_messages; sweep_expired_approvals() runs every 30min; POST /pro/agents/{did}/acp-policies; GET/PATCH/DELETE policies; GET /pro/agents/{did}/approvals; POST /pro/approvals/{id}/approve + reject.
v0.8.3	2026-05-11	Network page trust dimension fix — eliminated '??' and cross-page inconsistency. Root causes: (1) network.js fetched /agents/{did}/trust-score without ?detailed=true so dimensions were absent in every response; (2) cache stored dimensions=null but re-fetch guard checked ===undefined (never matched null) so retry never fired; (3) N individual API calls at boot (one per agent) instead of one bulk call. Fixes: prefetch loop replaced with single /pro/trust-scores call (all owned agents, dimensions included, one round-trip); panel-open re-fetch now uses ?detailed=true for external agents and /pro/agents/{did}/trust-score for owned agents; guard changed to ==null plus _detailFetched flag to prevent re-fetch loops; node colors re-drawn after bulk load returns.

v0.8.2	2026-05-10	Tasks UI — contract-first agent discovery. <code>tasks.html</code>: replaced manual 'To Agent DID' text input + hidden discover panel with a unified capability contract search flow. Users now type a capability keyword (e.g. <code>web_search</code>, <code>summarize</code>) → results come from <code>GET /capability-contracts/search?capability=X</code> (public, no auth needed) which returns agents with verified/challenge_passed contracts sorted by status → click Select → agent chip auto-fills DID + capability + SLA/pricing from the contract. Trust scores loaded async per agent (<code>GET /agents/{did}/trust-score</code>) and shown inline in results. If contract has <code>input_schema</code>, task-inputs textarea pre-populated with required fields and schema field hints shown below. On Send, capability comes from selected contract (not a manual field). Fallback: if no results with status filter, retries without it to show challenge_passed/pending agents. No more manual DID entry anywhere in the send flow.
v0.8.1	2026-05-09	Codex security review — 4 findings fixed. P3/multibase: <code>identity.py</code> <code>public_key_to_multibase()</code> added (<code>z + base58btc + varint 0xED01</code>); <code>Agent.create()</code> now emits correct <code>publicKeyMultibase</code>; <code>Agent.load()</code> corrects legacy base64 values on read; <code>did_document.py</code> detects non-z-prefix values in <code>stored_vms</code> and re-derives from authoritative <code>public_key</code>; <code>register_agent()</code> INSERT now populates <code>verification_methods</code> column with correct multibase on every new registration. P2/status drift: <code>eu_compliance.py</code> lines 274/408/477 updated from <code>status='active'</code> (non-existent) to status IN ('challenge_passed', 'pending_verification', 'verified'); <code>server_pro.py</code> MCP tool <code>_chat_search_contracts</code>: status enum updated, default changed 'active' → 'verified', legacy 'active' alias maps to all published statuses; <code>capability_type</code> → <code>capability</code> column fix in SQL queries and tool schemas; backward-compat kwarg aliases on <code>_chat_get_contract/</code> <code>_chat_log_capability_call/</code> <code>_chat_submit_attestation</code>. P1/federation: <code>federation.py</code> <code>PeerRequest.public_key</code> made required (was Optional); <code>add_peer</code> validates key is 32-byte Ed25519 before storing; <code>_verify_peer_response()</code> new helper performs Ed25519 sig verification; <code>_fetch_from_peer</code> now REQUIRES signature — unsigned responses rejected (not passed through). P1/encryption: <code>server_pro.py</code> <code>_encrypt_api_key()/_decrypt_api_key()</code> using Fernet (AES-128-CBC+HMAC) with 'fernet:' prefix sentinel for backward compat; <code>AGENTID_KEY_ENCRYPTION_KEY</code> env var; all 6 write paths encrypt on save; all 6 read paths decrypt before use; existing plaintext records read correctly without key rotation event.
v0.8.0	2026-05-09	Week 1 implementation — security hardening, crypto-agility, canonical SDK API, mcp-agentid wedge. SECURITY (Block 1): crypto-agility envelope on all <code>sign()</code> functions (<code>algSuite/version/params/signature</code> dict); TypeScript timestamp fix (<code>Date.now()</code> → <code>Math.floor/1000</code>); signer field standardised to 'signer' across Python/TS/Go/internal SDKs; path traversal fix in <code>registry._key_path()</code> (re.sub sanitisation + containment assertion); <code>nonce+issued_at</code> added to capability contract signed body; internal agentid-pro Node/Python SDKs updated with envelope + signer field. MULTI-KEY DID (Block 2): <code>AgentDocument.verification_methods</code> list[] per W3C DID Core; <code>did_document.py</code> accepts list[public_keys] and emits multiple <code>verificationMethod</code> entries; DB migration adds <code>verification_methods</code> JSONB column + GIN index. CANONICAL API (Block 3): <code>public_api.py</code> adds 5 missing symbols: <code>Receipt</code> (signed result envelope), <code>TrustScore</code> (typed trust response with <code>.fetch()</code>), <code>RemoteAgent</code> (read-only discovered agent handle), <code>signed()</code> (decorator factory), <code>verify()</code> (top-level shorthand with <code>trust_min + max_age_seconds</code>), <code>find()</code> (capability discovery with trust filter), <code>attest()</code> (peer attestation); <code>__init__.py</code> rewritten to export exactly 8 symbols: <code>Agent</code>, <code>signed</code>, <code>verify</code>, <code>find</code>, <code>attest</code>, <code>RemoteAgent</code>, <code>Receipt</code>, <code>TrustScore</code>. TEACHING ERRORS (Block 4): <code>http_registry.py</code> <code>_raise_with_context()</code> wraps all <code>raise_for_status()</code> calls with URL, operation, and actionable remediation hints; 409 conflict raises <code>ValueError</code> with <code>load()</code> guidance; <code>agent.py</code> missing-key messages include <code>Agent.load()</code> and <code>Agent.create()</code> instructions. TRUST SCORE API (Block 5): <code>public_trust_score()</code> accepts <code>?detailed=true</code>; default response includes <code>score/level/top_3_issues/trust_brief</code>; <code>_compute_top_3_issues()</code> scans breakdown for biggest penalty gaps across D1-D5, sorts by impact, returns top 3 as human-readable strings with D-dimension labels. CI (Block 6): <code>agentid/.github/workflows/ci.yml</code> runs <code>pytest</code> on push/PR; 16 tests in <code>test_cross_sdk.py</code> cover envelope format, timestamp seconds, path traversal containment, 8-symbol API, backward-compat bare strings, DID consistency check, <code>@signed</code> decorator. MCP-AGENTID (Block 7): new <code>mcp-agentid/</code> package — <code>decorator.py</code> (<code>@secure</code> with <code>trust_min/capabilities/audit/sign_response</code>), <code>audit.py</code> (AuditLog thread-safe JSONL with <code>tail()</code>), <code>trust.py</code> (<code>get_trust_score</code> 5-min cache, <code>check_trust</code> with <code>PermissionError</code> + <code>top_3_issues</code>), <code>pyproject.toml</code> (PyPI-ready, <code>mcp-agentid</code> name), 17 tests.

v0.7.0	2026-05-08	Project Meridian — 10-agent open-world cooperation experiment. scripts/meridian_setup.py: 10 specialist agents (oracle, nexus, cipher, loom, scribe, forge, herald, aegis, muse, judge) each with distinct personalities, 3 capability contracts, and system prompts that name their capability gaps and reference find_agent_for_task() to discover collaborators. 30 total contracts across 10 agents covering: statistical-analysis, anomaly-detection, intelligence-gathering, web-research, threat-assessment, systems-mapping, cascade-failure-modeling, report-writing, mathematical-validation, plain-language-synthesis, publication-risk-assessment, hypothesis-generation, evidence-synthesis, and more. Seed messages loaded from New project/seed_messages.py — each agent given a starting state of incomplete information that forces cooperation: oracle has the numbers but no context, nexus has incidents but no statistics, cipher has indicators but no data, loom has a model but no real data, scribe has templates but no content, forge has no work yet, herald has no report, aegis hasn't seen the report, muse challenges the human-adversary assumption, judge waits for 3+ independent submissions. All 46 find_agent_for_task() calls verified to resolve to registered capability contracts. scripts/meridian_observe.py: live documentary observer — polls all agent inboxes and group rooms, tracks contact graph + first-contact events, generates narrative snapshots, detects completion phrases ('meridian assessment', 'final verdict', etc.), writes meridian_log.txt, shows ANSI color output with per-agent color coding.
v0.6.3	2026-05-08	Brain improvements: MCP sampling priority chain fix, human group sender, depth cap (commit 9114fb7). mcp_server.py: _owner_sessions dict (owner→api_key), register_mcp_session(api_key, owner=") now also stores owner mapping, get_active_session_for_owner(owner) returns active api_key or None, cleanup_stale_sessions() also prunes _owner_sessions. server_pro.py _McpSseHeaders: SSE connect now does DB lookup to find owner from key_hash, calls register_mcp_session(api_key, owner=_sse_owner) so brain can resolve sessions by owner. _auto_respond_group_bg: Priority 0 MCP sampling block (before Anthropic/OpenAI): get_active_session_for_owner(agent_owner) → sample_reply_via_mcp_sync if active. _agent_brain_cycle: same Priority 0 block for 1:1 judgment calls. _auto_respond_bg (1:1): fixed latent bug — was calling is_mcp_session_active(agent_owner) with owner email instead of api_key; now uses get_active_session_for_owner correctly. Depth cap: _auto_respond_group_bg_depth >= 3 → _depth >= 2 (stop after 2 rounds). group_chat.py POST /pro/chat/rooms/{id}/messages: allow from_did='human:{owner}' so owners can post to group rooms directly (skips agent-membership check, verifies owner matches, requires ≥1 active agent in room); AI members auto-reply normally.
v0.6.2	2026-05-08	PageRank batch job over endorsement graph (3.4, commit 17b5c9b). NEW MODULE pagerank.py: agent_pagerank table (did, pagerank FLOAT 0-1, computed_at). run_pagerank(): power-iteration PageRank, damping=0.85, max 30 iters, convergence=1e-6. Nodes=all DIDs in capability attestations (cross-owner); edges=reviewer→target weighted by verdict (confirmed=1.0×, partial=0.5×, not_dem=0×) × reviewer_trust_at_time/50 (neutral=1.0×, high-trust=2.0×). Handles dangling nodes; normalises ranks to 0.0-1.0. get_pagerank_score(did) returns 0.0-1.0 for D3 integration. GET /agents/{did}/pagerank + POST /internal/pagerank/run endpoints. server_pro.py: pagerank schema init at startup; nightly batch after snapshots. trust_score.py: D3 max raw 20→25, pagerank_score 0-5 pts feeds D3; pagerank breakdown field; endorsement graph guidance in trust brief.
v0.6.1	2026-05-08	POLP enforcement at API + MCP layer (3.6/5.2, commit ab8f778). capability_contracts.py: _POLP_ENFORCE flag (POLP_ENFORCE env var, default=0=audit-only), _check_polp(callee_did, capability, caller_did) helper — looks up scope_limits from agents table, exact+prefix match ('research' permits 'research-summary'), fails open on DB errors, records scope_violation trust event (-5 pts D4) on out-of-scope detection. log_call_endpoint (Layer 2): POLP check pre-log; POLP_ENFORCE=1=HTTP 403 block, default=allow+scope_warning in response. GET /agents/{did}/scope-check?capability=... endpoint for direct POLP query. mcp_server.py: check_agent_scope(did, capability) read-only MCP tool; added to MCP instructions for pre-delegation scope verification.

v0.6.0	2026-05-08	Trust Score Phase 4 — D2/D3/D6 enrichment + trust brief D6 fix (commit 8432e63). D2 max raw 70→80: agent_uptime_days table (one row/agent/day on each heartbeat ping), uptime_score 0-10 from 90-day active_days/90*10, sla_breach_penalty -2/expired-cross-owner-task (90d, max -10). D6 full 5-signal model per capability: Signal1=peer attestations weighted by reviewer_trust_at_time, Signal2+3=call log success rate + sla_compliance_score from capability_contracts, Signal4=task completion rate per capability (agent_task_requests GROUP BY capability), Signal5=per-capability complaints via agent_complaints.category LIKE 'capability:%'. Each capability now returns {score:0-100, signals:{attestation,call_success,task_completion,complaints}, detail:{call_count,avg_latency_ms,task_rate,open_complaints}}. accountability.py: category TEXT column + index on agent_complaints (idempotent ALTER). liveness.py: agent_uptime_days table + ping handler records active day (idempotent). _generate_trust_brief(): fixed to handle dict-format capability_trust values, CAPABILITY TRUST section shows per-signal breakdown, red-flag + guidance sections updated.
v0.5.9	2026-05-08	Trust Score Phase 3 — 6-dimension hardening (Part D continued). COMMIT 1 (5c7023a) trust_score.py: complaint density -3/complaint(90d,max-20)+resolution bonus→D3, graduated badge D5 (tier1=15/tier2=20/tier3=25 based on contracts+clean record), signing compliance 0-10 pts from outbound msgs 30d→D1, D1 raw max 25→40, key_rotation_history table + rotation_bonus 0-5 pts, decay-weighted D4 (0-7d=1.0×,7-30d=0.7×,30-60d=0.4×,60-90d=0.2×, 90d window). key_rotation.py: key_rotation_history table created, confirm_rotation logs completed rotations. COMMIT 2 (57b703e): task_completion (0-10 pts, cross-owner 90d)→D2, D2 raw max 60→70; scope_violation_penalty (-5/violation 90d, max-15)→D4; dimension_snapshots.py _extract_dim_score() bug fix (dims are dicts not floats); server_pro.py _record_scope_violation() helper; mcp_server.py check_trust_for_action+get_trust_score now inject trust_context brief; public /agents/{did}/trust-score now includes trust_brief field. COMMIT 3 (96a33a8): scope_limits + human_oversight columns on agents table (idempotent ALTER), RegisterRequest accepts scope_limits (list[str]) + human_oversight (none advisory required always), UPDATE endpoint handles scope_limits/human_oversight from signed payload, get_scope_info(did) helper, DID document #scope-declaration service endpoint, D5: badge_gov(0-50)+contracts_gov(0-35)+scope_gov(0-15)=100 max. COMMIT 4 (cddabc2): DID-document consistency check: public_key_to_did(stored_key)==did? Mismatch hard-pins D1=0, did_consistency breakdown field, CRITICAL red flag in trust brief.
v0.5.8	2026-05-08	Trust Score Phase 2 — dimension history, SLA into D2, trust brief injection. NEW MODULE dimension_snapshots.py: agent_dimension_snapshots table (D1-D5+composite per day), take_snapshot_for_did(), take_all_snapshots() batch, compute_30d_trend(), compute_90d_volatility(), GET /pro/agents/{did}/dimension-history endpoint (snapshots+trends+volatility), POST /internal/snapshots/take-all (internal trigger), daily background job in server_pro.py (~25h interval). trust_score.py: SLA compliance score from capability_contracts.sla_compliance_score (0-10 raw pts) fed into D2 (max raw 50→60), sla_compliance in breakdown dict, dimensions now return {score, trend_30d, volatility_90d} objects, _dim_score() helper, _generate_trust_brief() updated for new dict-format dimensions with trend arrows. mcp_server.py: MCP_INJECT_TRUST_BRIEF env var (default=1), _fetch_trust_brief() async helper, send_message() appends recipient_trust_context to result, find_agent_for_task() parallel-fetches briefs for top-5 results and adds trust_context per agent.

v0.5.7	2026-05-07	Trust Score Phase 1 — 6-dimension decomposition (Part D of Strategic Review). <code>trust_score.py</code> now computes a 6-dimension vector alongside the composite score: D1 Identity Integrity (age+revocation+deprecation, hard-pinned to 0 if quorum compromise), D2 Operational Reliability (verification rate+volume+liveness), D3 Network Reputation (cross-owner attestations + positive feedback), D4 Behavioral History (interaction events adjusted by complaints), D5 Governance (badge graduated to 50 pts + contracts up to 50 pts, removes binary cliff), D6 Capability Trust (per-capability map from <code>capability_attestations</code> GROUP BY capability). Added <code>_generate_trust_brief()</code> — server-side LLM context string with dimensions, red flags, and guidance. All 3 trust-score endpoints now return dimensions field. Pro authenticated endpoint returns full <code>trust_brief</code> string. Network map frontend auto-unlocks 6-dimension section when dimensions arrive (no frontend change needed). Network map bug fixes: ego re-centre translate fix, exit-ego frozen layout fix (saved position snapshot), <code>_suppressNextClick</code> flag for click/mouseup event ordering, Barnes-Hut $O(n \log n)$ physics, <code>NODE_CAP</code> raised to 800.
v0.5.6	2026-05-07	Brain improvements + tasks auth fix + network ego re-centre. MCP sampling promoted to Priority 1 in <code>_auto_respond_bg</code> : when Claude Desktop is connected the reply is generated there using full session context + optional Tavily enrichment; falls back to Anthropic/OpenAI API key automatically. Group brain history window increased from 20 to 30 messages for richer context. Judgment cache added: YES/NO speak decisions cached 10s per (room_id, msg_hash, agent_did) to prevent duplicate LLM calls on retries. <code>tasks.html</code> auth: fixed wrong localStorage key (<code>agentid_key</code> → <code>agentid_persisted_key</code>), added <code>credentials:include</code> to all fetch calls, fixed inline early-show script. Network ego re-centre: clicking a non-center node now translates the existing layout rather than swapping positions — previously visible nodes slide to new positions, brand-new nodes placed on perimeter ring, no jarring overlap.
v0.5.5	2026-05-07	Network map click + physics fix. Added 6 px drag-detection threshold (mouse) / 10 px (touch) so single clicks are always registered — previously any sub-pixel mouse tremor set <code>moved=true</code> and silently ate the click (required triple-click). Same threshold prevents accidental node teleportation which reheated physics and scrambled the graph layout when clicking non-center agents. Also calls <code>stopSimulation()</code> at the top of <code>enterEgoMode()</code> to cancel stale RAF loops before swapping the node-set, and clears <code>_panMoved</code> so the first click after ego-mode entry is never blocked.
v0.5.4	2026-05-07	Security audit + dashboard premium overhaul. 13 codebase findings fixed: broken admin gate (<code>accountability.py</code>), SQL f-string injection (<code>trust_routing.py</code>), SSRF redirect bypass (<code>identity_binding.py</code>), CORS missing PUT, threading locks for <code>_bypass_tokens</code> + <code>_search_usage</code> + <code>_SESSION_CACHE</code> , session sliding-window fix, async long-poll for task status, <code>_account_age_days</code> uses users table. EU AI Act Compliance Mode (P8): <code>eu_compliance.py</code> , 6 endpoints, risk tiers, human review queue, bypass tokens, SHA-256 hash chains, compliance PDF report, DID document risk tier service entry. Dashboard premium: custom scrollbar, card elevation + hover lift, stat card sparklines, sidebar pill-nav + avatar footer, entrance animations, dark mode glass morphism. Preferences tab in Settings: 9 toggles + 2 selects, localStorage persistence, compact mode. Network map: force-directed physics (Coulomb+Hooke+gravity), golden angle spiral layout, gradient edges, trust badge overlay, detail panel with trust score SVG ring, capability chips, activity sparkline, top connections.
v0.5.3	2026-05-07	Task API friction fix: removed <code>require_pro()</code> gate (free tier can use task API), made capability contracts optional (tasks succeed without a pre-published contract; schema validation only fires when a contract exists). Zero config required — any registered agent can send tasks to any other registered agent.
v0.5.2	2026-05-07	Agent-to-Agent Task API (<code>agent_task_requests.py</code>): 5 REST endpoints, <code>agent_task_requests</code> table, state machine (pending→accepted→running→completed/failed), atomic UPDATE-WHERE transitions, jsonschema input validation, capability auto-routing, expiry sweeper, 6 webhook events. Brain: <code>execute_task</code> action + tool loop (Anthropic+OpenAI). <code>auth.py</code> : <code>require_agent_ownership()</code> . Reviewed by 4 agents pre-build.

v0.5.1	2026-05-07	Group chat brain integration: pleasantry filter, burst cap (4 msg/2 min), Anthropic + OpenAI reply paths, web_search in group, dedup guard, judgment prompt defaulting to YES for substantive messages. Group view flicker fix (renderFeed early-return). DM brain: agreement-loop token expansion, _brain_send_reply dedup, OpenAI tool_call_id normalization.
v0.5.0	2026-05-06	MCP bridge (MCPSession + MCPProxyTool), multi-provider layer (AnthropicProvider, OpenAIProvider, GeminiProvider), agentic ReAct judgment loop (8 rounds), WebSearchTool, FetchURLTool, AgentBrain orchestrator, OnChangeTrigger, 71 tests total.
v0.4.0	2026-04-xx	AgentBrain core — Perception layer (Git, File, API), Judgment engine, Action executor, BrainMemory, IntervalTrigger, DailyTrigger.
v0.3.0	2026-03-xx	Runtime SDK — AgentRuntime long-polling loop, WebhookTransport, HMAC-SHA256 verification, CLI (agentid-runtime).
v0.2.0	2026-02-xx	Auto-reply multi-provider — Anthropic + OpenAI providers, diagnostic logging, MCP sampling (Claude Desktop as reply brain), priority chain.
v0.1.0	2026-01-xx	Initial scaffold — server, auth, messaging, MCP server, agent registration, DID system.

Document Index

All AgentID technical documents are stored in:

```
/Users/bereket/Documents/AgentID_Docs/
```

Filename	Topic	Status
00_Master_Tracker.pdf	This document — full build tracker	Current
01_Autonomous_Search_ReAct.pdf	When/why agents search — ReAct pattern, two-phase detection	Current
AgentID_Runtime_Layer_Plan.pdf	Runtime SDK design (long-poll, webhook, CLI)	Older — see SDK code
AgentID_Auto_Register_Webhook.pdf	Webhook auto-connect design	Older — see runtime/webhook.py
AgentID_Admin_Guide.pdf	Admin operations guide	Older
AgentID_Strategic_Review.pdf	Strategic vision document	Older
AgentID_Todo_List.pdf	Original 85+ task checklist	Superseded by this doc